Optimizing Compiled Verilog

Randy Allen and Michael McNamara
Chronologic Simulation

Abstract

While optimization of traditional programming languages is reasonably well understood, optimization of hardware description languages is much less so. Hardware description languages provide both a different set of problems and a different set of opportunities for optimization. On the problem side, hardware models tend to be very large compared to software applications. As a result, the optimizer (which is typically a memory hog even on modest applications) must focus carefully on memory usage. On the opportunity side, hardware models tend to have much simpler control flow than do software applications. This permits a more efficient set of analysis techniques—techniques that are not possible in compilers for high level languages. This paper discusses the ways in which traditional compiler optimization techniques have been adapted in a compiler for Verilog.

Introduction

Code optimization has been an important consideration in compiler construction since the development of the first FORTRAN compilers. Even the most sophisticated code generator will produce code that can use some improvement, and most code generators are not that sophisticated. Because of this, intensive research has been focused on compiler optimization. The result of this research is a number of general mechanisms for improving code generated by compilers for traditional programming languages. Reasonable production compilers now include global optimizations (such as constant propagation, dead code elimination, and sophisticated register allocation), local optimizations (such as common subexpression elimination and constant folding), and loop optimizations (such as strength reduction and code hoisting). The theoretical basis for these optimizations involves treating the control flow of a program as a lattice framework over which data is propagated. There are many different techniques for propagating the data, but the mathematical underpinnings are all based on discrete lattice models.

Even though optimizers have strong theoretical underpinnings, their development is still somewhat more an art than a science. There are a number of reasons for this:

1. Edge cases. While theory covers the basics of optimizing a language, almost every language contains edge cases that don't quite fit theory. In FORTRAN, the edge cases come from EQUIVALENCE and COMMON statements and various control flow constructs that give difficulty. In C, pointers and aliasing provide the key problems.
2. Memory consumption. In the ideal, theory can support a wide range of optimizations across all user variables. In practice, theory must be tempered or restricted, as an unrestrained optimizer unleashed on a moderate program will easily consume more memory than is available.
3. Compile-speed. Just as optimizers can consume large amounts of space, even the most rapid algorithm in theory can require excessive amounts of compile-time. Choosing the correct algorithm for analysis is not a simple matter of selecting the theoretically fastest available.
4. Profitability. Determining whether a transformation actually improves a program's performance is not so simple a task as it might seem. Optimizers must be strongly concerned with avoiding performance degradations.

Because of these factors, developing an optimizer for programming languages is a careful balance of theory and pragmatism.

As important as optimizers are to traditional compilers, they are even more important to compilers of hardware description languages. While the arithmetic performed in hardware descriptions is usually simple, the use of multi-valued logic complicates the resulting operations. To illus-

0-8186-5655-7/94 $03.00 © 1994 IEEE

Authorized licensed use limited to: University of Michigan Library. Downloaded on March 20,2026 at 02:38:29 UTC from IEEE Xplore. Restrictions apply.

trate this complication and potential simplifications, consider the following Verilog fragment:

```
reg[31:0] A,B,S;

S = A & B;
```

This simple "and" is defined, given Verilog's 4 value semantics, as follows. Assuming a Verilog register is stored into machine integers, one holding the "data" fields, and the other the "control" fields, then this "and" is equivalent to:

```
struct { int c,d; } S,A,B;

S.d = (A.d | A.c) & (B.d | B.c);
S.c = S.d & (A.c | B.c)
```

This code, when compiled, turns into code similar to the following (represented in C):

```
typedef unsigned int U;
a = * (U *) (ip+88); /*A's cntl*/
b = * (U *) (ip+96); /*B's cntl*/
d = (a | (* (U*) (ip+92))) &
   (b | (* (U*) (ip+100)));
* (U *)(ip+232) = d & (a | b);
* (U *)(ip+236) = d;
```

This is a large amount of code to implement a simple "and" of two vectors. The majority of the code is checking for the presence of "x" and "z" inputs; the first two loads are of control fields holding 1's if the associated data fields represent "invalid" inputs and 0's if the data fields hold valid 0's or 1's. Should it be known that neither A nor B have any "x" or "z" bits, (which is the expected case once a design gets out of reset), the fragment reduces to

```
* (U *)(ip+232) = 0;
* (U *)(ip+236) = *(U *) (ip+92)
          & *(U *) (ip+100);
```

The job of an optimizer is to derive information about a program and use that information to produce a faster executable for that program. Obviously, Verilog provides a number of opportunities for an optimizer to exploit.

Just as an optimizer for a software application language is a careful balance of theory and pragmaticism, an optimizer for Verilog must be carefully balanced. On one hand, hardware models usually involve a wider set of inputs and outputs than do application programs. Modeling these inputs and outputs require memory, causing more intensive memory usage than application programs. Since optimizers are memory hogs to begin with (requiring memory proportional to the compiled program's memory requirements), optimizers for hardware models must be carefully tuned. On the other hand, hardware models have simpler control flow than application programs, providing the opportunity for using faster optimization techniques than are usually employed in optimizers for programming languages.

This paper discusses the balances struck in developing an optimizer for a Verilog compiler. The next section presents a brief description of the intermediate representation used, since the form that optimizations take is highly dependent on the structures used to represent the program. Following that is a section developing the important optimizations necessary for a Verilog compiler. Section 4 lists the balances struck for local optimizations; section 5 describes the theoretical considerations behind global optimizations. The final section summarizes these considerations, as well as presenting potential future directions.

Intermediate Representation

Intermediate representations can range from very high level (that is, a representation that is very close to the source language being compiled) to very low level (that is, a representation that is very close to the target machine's instruction set). Given the goal of generating good code for a number of different architectures (mostly, but not entirely, RISC architectures), a natural choice for intermediate representation is one that is fairly close to RISC instruction sets. The VCS™ (Chronologic Simulation's Verilog compiler) optimizer works at this level of representation.

The basic unit of the intermediate representation is a MOP (abbreviation for Machine OPeration). MOPs are intended to map one-to-one (or as closely as possible) to machine instructions; examples are add, subtract, load, store, and shifts. Each MOP holds three operands: two input and one output, again mirroring common instruction sets. MOPs are grouped together by basic blocks, where a basic block is a maximal set of MOPs such that any one member of the block is executed if and only if all other members are also executed. In other words, a basic block is the largest set of MOPs that contains no change in control flow—control flow changes are represented as edges between basic blocks.

MOPs operate on two basic kinds of data: memory locations (which are primarily used to hold source level symbols and associated information) and compiler temporaries (intended to provide an infinite, symbolic register set).

Authorized licensed use limited to: University of Michigan Library. Downloaded on March 20,2026 at 02:38:29 UTC from IEEE Xplore. Restrictions apply.

Memory locations are all represented as a base register and an offset, as in the previous examples where the base register is "ip". By default, all user variables and control information are allocated as offsets from "ip", which points to statically allocated memory for a given module. Compiler temporaries are intended to provide scalar locations which hopefully map into the target architecture's register set. The single letter variables (a, b, and d) in the previous example are all compiler temporaries.

Frontends generate simple intermediate code on a local basis, making no attempt to reuse results or minimize memory access. That job is instead left to the optimizer.

Important Optimizations

A major part of the design process for an optimizer is deciding which optimizations are most important and which are less important. Typically, a few optimizations will provide a very high return in terms of improved execution speed while the remainder provide only marginal improvements. Given that compile-speed and compile-space are always limitations, this means the best returns are yielded by focusing on a few, important optimizations.

The sample Verilog fragment presented earlier illustrates several opportunities for optimization. The key to those optimizations involves recognizing that the control fields of the incoming variables are always zero—that is, at run-time, the fragment can be correctly executed as:

```
typedef unsigned int U;
*(U *)(ip+88) = 0; /*A is valid*/
*(U *)(ip+96) = 0; /*B is valid*/
a = * (U *) (ip+88); /*A's cntl*/
b = * (U *) (ip+96); /*B's cntl*/
d = (a | (* (U*) (ip+92))) &
   (b | (* (U*) (ip+100)));
* (U *)(ip+232) = d & (a | b);
* (U *)(ip+236) = d;
```

Once that fact is known, simple constant propagation will produce

```
typedef unsigned int U;
*(U *)(ip+88) = 0; /*A is valid*/
*(U *)(ip+96) = 0; /*B is valid*/
a = 0; /*A's cntl*/
b = 0; /*B's cntl*/
d = (0 | (* (U*) (ip+92))) &
   (0 | (* (U*) (ip+100)));
* (U *)(ip+232) = d & (0 | 0);
* (U *)(ip+236) = d;
```

Since a number of the variables involved are constants whose values have been substituted forward, the variables proper are no longer necessary. Assignments to the variables can be eliminated as dead code, yielding

```
typedef unsigned int U;
d = (0 | (* (U*) (ip+92))) &
   (0 | (* (U*) (ip+100)));
* (U *)(ip+232) = d & (0 | 0);
* (U *)(ip+236) = d;
```

Finally, straightforward constant folding and simplification will convert the fragment into the simple fragment that appears obvious from the original Verilog source:

```
typedef unsigned int U;
d = *(U *)(ip+92) &
    * (U*)(ip+100);
* (U *)(ip+232) = 0;
* (U *)(ip+236) = d;
```

As should be evident from this simple example (which is representative of large amounts of Verilog programs), the key factor in optimizing the fragment is discovering that the control fields are zero. In general, this discovery requires complicated analysis—some combination of interprocedural analysis, higher level Verilog analysis, or user assertion. While it is desirable to solve the general problem in the long run, in the short run, this information is often available in local cases by a simple analysis of the code. In any case, once the information is available, it is clear that the key global optimizations are constant propagation and dead code elimination.

Given a local strategy for code generation that attempts no memory reuse, it should also be evident optimizations focused on recognizing and exploiting memory reuse are important. In particular, users tend to work with the same variables repeatedly in a section of code. Code generators rarely recognize this reuse; they load the locations they need at the beginning of a unit and store everything changed at the end of a unit. There are three ways of improving this reuse: eliminating common subexpressions, changing variable allocations, and doing good register allocation.

Common subexpression elimination (cse) is a well known optimization that comes in both local and global forms. Just as in traditional compilers, the important case in a Verilog optimizer is local cse. Users tend to work with a number of common variables in a section of code, then move on to different variables. Local cse eliminates the redundancies arising from this type of operation. There is

Authorized licensed use limited to: University of Michigan Library. Downloaded on March 20,2026 at 02:38:29 UTC from IEEE Xplore. Restrictions apply.

rarely much benefit to be gained from searching globally for redundant operations, and the search time is expensive, making global cse of little benefit.

Variable allocation is concerned with whether user variables are allocated statically (via static storage) or dynamically (off the stack). By default, variables are allocated to static storage. This means that if an optimizer sees a store to a variable at the end of a procedure, and that store is never fetched again within that procedure, the optimizer must leave the store in place. The reason is that the next invocation of the procedure could use that value. Dynamic storage, however, disappears at the end of an invocation, so that an optimizer could safely delete such a store to stack storage. Changing static variables to dynamic variables is a simple global optimization based upon determining whether a variable is used before being set within a procedure. When the optimizer can determine that a variable is always defined before it is used, the optimizer can safely move that variable to dynamic storage.

Loop optimizations are probably the most important optimizations to perform in optimizers for traditional programming languages, but they are relatively unimportant in a Verilog compiler. Hardware models tend to have very little feedback in the form of loops, and those few loops that do appear are not extremely amenable to optimization. As a result, loop optimizations can be completely ignored with little adverse impact on the optimizer.

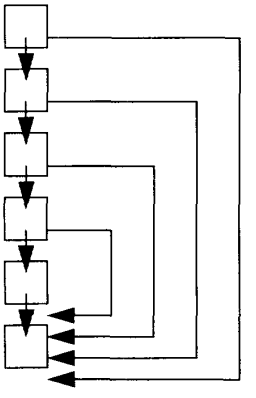
Summarizing, the optimizations that are the most important in a Verilog compiler are the global optimizations constant propagation, dead code elimination, register allocation, and variable allocation, as well as local common subexpression elimination. Loop optimizations are not very important.

Local Optimizations

Common subexpression elimination is a well known optimization, which is typically implemented by using a hash table to keep track of available expressions and their locations. The VCS™ implementation is similar, with only a couple of wrinkles necessary for Verilog. While cse tracks all expressions, the really important ones are loads and stores—it may or may not be beneficial to eliminate a redundant "and" operation (which takes only one cycle on most machines). It is absolutely essential to eliminate unnecessary memory accesses. Fortunately, given that the intermediate representation reduces all variables to offsets from a single pointer, hash functions focused on loads and stores are extremely simple to come up with—the most obvious being using the offset itself as a direct access into a table.

Even though hash functions are simple to generate, a lot of caution is necessary for memory usage. User variables are allocated statically across an entire module, which usually consists of several subroutines. Since there is no attempt to optimize accesses via subroutines (such a problem is clearly NP-complete), it is very possible for a subroutine to access only two memory locations—one at a low offset such as 4 and one at a high offset such as 100,000. Cse must be carefully implemented to adequately handle this sparsity of access. In most programming languages, access is guaranteed to be dense, allowing simpler cse.

One other wrinkle is important in cse for Verilog. The nature of many hardware logic operations is to do a little bit of work and test some results of that work. If those results are "OK", then that operation is done; otherwise some more work is done and another test implemented. As a result, control flow often ends up looking like



Normally, cse would only operate on one basic block—a box in the diagram. However, given the predominance of this form of control flow, a simple modification to cse allows it to operate on an "interval" (a single-entry, multiple-exit region). The first 5 regions above comprise a single interval, and cse's effectiveness is greatly increased for Verilog when it operates on such larger regions.

Dataflow Techniques

Global optimizations are all based on information garnered across an entire program. The basic theory behind all global optimizations involves treating propagating information from basic block to basic block along the control flow of the program until a fixed point is reached—

Authorized licensed use limited to: University of Michigan Library. Downloaded on March 20,2026 at 02:38:29 UTC from IEEE Xplore. Restrictions apply.

nothing changes when more information is propagated. Simply stated, this propagation is locating the point on a lattice on which the program "resides". There are several techniques for propagating information from block to block. The simplest algorithm, the "iterative algorithm", does exactly the simple propagation described above. While the iterative algorithm is theoretically quadratic in the number of basic blocks, it is extremely simple to program. More complex methods involve a pass of combining basic blocks into larger regions (at the same time, combining the information they contain) until global information is obtained, then reversing the process to distribute the information. Linear time algorithms are possible under this process, although they are complex to program.

In practice, the simple iterative algorithm is very effective, and its quadracity rarely arises. However, a straightforward application of the iterative algorithm does not work well with hardware models. The primary problem is memory requirements—the iterative algorithm requires 3 bits for every memory location tracked in the program. In even a moderate sized model, this can amount to hundreds of megabytes of space.

Fortunately, the form of control flow graphs in hardware models permits a simple modification of the iterative algorithm to work well without exhausting memory. Since hardware models usually demonstrate forward flow, with very few loops, they can easily be decomposed into large single-entry, single-exit regions (also known as "hammock graphs"). Dataflow within a single hammock is obtained via the iterative algorithm; dataflow across hammocks can also be obtained in general via the iterative algorithm. However, applying the iterative algorithm across hammocks is unnecessary, and also requires extensive amounts of memory. Given the simple forward flow, information can be propagated from hammock to hammock in a forward-flow pass. This allows triangular bit vectors to be used, cutting total memory requirements by a constant factor. While constant factors are usually not significant in optimizers, this one is; it in general cuts memory requirements from hundreds of megabytes down into the small tens of megabytes.

Summary

Developing an optimizer for any language is a careful balance of theory and pragmaticism, and Verilog is no exception. Hardware models have exhausting memory requirements, which forces optimizers to be parsimonious in their use of memory. On the other hand, the simple control flow graphs encountered allow algorithm simplifications that cut both execution time and memory requirements.

At one level, the thesis of this paper is that optimizers are important for compiled Verilog, and that they can be effectively developed. However, that is not a very surprising result; one would hope that any reasonable language can be effectively compiled—if it cannot, it is probably not a reasonable language. A more significant and surprising thesis is that such an optimizer can be developed in a relatively small amount of time (the VCS™ optimizer took roughly half an engineer-year) and that it can strongly outperform more general optimizers that are less focused.

The VCS™ native code generator produces code using much less memory, using much less time, and code that runs significantly faster than that produced by its unoptimized C output passed through commercial C compilers with full optimization enabled. Given that the optimizers on these compilers have many engineer-years of investment in their development, the general lesson is that focusing on a small set of highly effective optimizations is probably a better road to a good optimizer than applying a very broad set.

Authorized licensed use limited to: University of Michigan Library. Downloaded on March 20,2026 at 02:38:29 UTC from IEEE Xplore. Restrictions apply.